

Manual operativo de datos — Ecosistema de Automatización IA

Matías Martínez · Entrega Final · Base: "Centro de Comando Contenidos" (Airtable)

1. Esquema de tablas y relaciones

La base actúa como la memoria única del sistema: n8n nunca guarda estado propio, todo vive en Airtable. Son 4 tablas vinculadas entre sí:

Tabla	Rol	Campos	Relaciones
Contenidos	Tabla de control del pipeline: cada fila es una pieza de contenido con su ciclo de vida completo.	Idea semilla (texto) · Borrador (texto largo, lo escribe la IA) · Estado (selección única: Generando / En revisión / Aprobado / Rechazado / Publicado) · Aprobado (checkbox, semáforo HITL) · Última ejecución (hora de última modificación, automática) · Publicado el (fecha/hora, la estampa n8n al publicar)	← Log de errores (Registro afectado) ← KPIs (Piezas)
Directrices	Base de conocimientos privada (RAG) con información atómica: una regla de marca por registro.	Tema (texto: Tono, Público, Estructura, Hashtags, Prohibido) · Directriz (texto largo)	La lee el flujo en cada ejecución (búsqueda completa) y la inyecta al prompt de sistema.
Log de errores	Registro de fallos que alimentan las rutas de error del flujo.	Evento (texto) · Nodo (texto: dónde falló) · Detalle (texto largo: mensaje de la API) · Registro afectado (vínculo → Contenidos) · Fecha (hora de creación, automática)	→ Contenidos (vínculo directo a la pieza que falló: trazabilidad completa)
KPIs	Panel de control del dashboard.	Panel (texto) · Piezas (vínculo → Contenidos) · Errores (vínculo → Log de errores) · Total piezas (count) · Total errores (count) · Tasa de errores (fórmula: errores/piezas %)	→ Contenidos y → Log de errores (los contadores y la tasa se calculan sobre los vínculos)

Los estados de Contenidos funcionan además como candados de sincronización: el flujo solo toma piezas en "Generando" y solo publica piezas "En revisión" con el checkbox aprobado. Una pieza "Publicada" o "Rechazada" nunca vuelve a procesarse, lo que elimina los bucles infinitos por diseño.

2. Esquemas JSON de transferencia entre integraciones

a) **Airtable → n8n (lectura de una pieza).** El nodo de búsqueda devuelve cada registro aplanado; los campos se mapean como variables dinámicas (nunca datos fijos):

```
{
  "id": "reck6DsGLvqpE1JbC",
  "createdTime": "2026-09-21T14:02:11.000Z",
  "Idea semilla": "5 señales de que tu pyme necesita un dashboard de control",
  "Estado": "Generando"
}
```

b) **n8n → OpenAI (petición de redacción).** El prompt es 100% dinámico: el bloque de sistema se construye con las directrices leídas de Airtable y el de usuario con la idea semilla del registro en curso:

```
{
  "model": "gpt-4o-mini",
  "max_tokens": 400,
  "temperature": 0.7,
  "messages": [
    { "role": "system", "content": "Eres el redactor de contenidos... directrices privadas:\n{{ directrices agregadas }}" },
    { "role": "user", "content": "Redacta un post de LinkedIn a partir de esta idea semilla: {{ Idea semilla }}" }
  ]
}
```

c) OpenAI → n8n → Airtable (guardado del borrador). La variable de respuesta se mapea desde **choices[0].message.content** y se escribe junto con el cambio de estado:

```
{
  "id": "{{ id del registro en curso }}",
  "Borrador": "{{ choices[0].message.content }}",
  "Estado": "En revisión"
}
```

d) n8n → WhatsApp Cloud API (publicación, solo tras aprobación humana). Mensaje de texto libre con número en formato internacional:

```
{
  "messaging_product": "whatsapp",
  "to": "569XXXXXXX",
  "type": "text",
  "text": { "body": "PUBLICADO ✅ Post aprobado y distribuido:\n\n{{ Borrador }}" }
}
```

e) n8n → Airtable (ruta de error). Cualquier fallo de API o dato faltante genera este registro, con el vínculo al registro afectado:

```
{
  "Evento": "Fallo del motor de IA",
  "Nodo": "IA: Redactar post",
  "Detalle": "{{ JSON.stringify(error) }}",
  "Registro afectado": ["{{ id del registro en curso }}"]
}
```

3. Sincronización

Toda la sincronización la gestiona el flujo: Airtable es la fuente de verdad, n8n lee y escribe estados, y ninguna integración habla con otra directamente. Si el flujo se detiene, el sistema queda en un estado consistente (las piezas conservan su estado y se retoman en la siguiente ejecución del trigger).